Politecnico di Torino

Master of Science (M.Sc.) in Computer Engineering

Academic Year 2025/2026

Graduation Session March 2026

Design and Implementation of a Cloud-Native Reactive Architecture

The Smart Travel Platform Case Study

Supervisor

Prof. Giovanni Malnati

Candidate

Federico Palazzi

Summary

Introduction and Research Area

The purpose of this thesis was to design, develop, and deploy Smart Travel, a fullstack, cloud-native web platform that empowers travel agencies to compose, market, and sell custom travel packages, while allowing customers to easily search for flights, accommodations, and activities. The primary objective was to solve the main challenges of distributed computing — namely data consistency across microservices, secure inter-service communication, network latency, and deployment automation. The solution implements state-of-the-art industry patterns such as the Backend-For-Frontend (BFF), the Transactional Outbox pattern, Change Data Capture (CDC), and Continuous Integration/Continuous Delivery (CI/CD) pipelines via Kubernetes-based orchestration.

System Architecture Overview

To achieve strict separation of concerns, the platform was decomposed into distinct microservices, each responsible for a specific domain:

- **Travel Catalog API:** Developed with Quarkus to manage travel packages, flights, accommodations, and activities.
- **User API:** Developed with Spring Boot to handle customer profiles and authentication data.
- **Orders API:** Developed with Spring Boot to manage the orders checkout and financial payment workflows.
- **Notification API:** Developed with Quarkus to asynchronously process and dispatch email alerts.
- **API Gateway (BFF):** Developed with Spring Boot to aggregate downstream data from microservices and to secure the API endpoints. It exposes a unified

GraphQL interface, abstracting away the underlying RESTful microservice topology and preventing tight coupling between the client and backend services.

Beyond the core backend services, the architecture integrates:

- **Database:** To guarantee decoupled persistence, each microservice operates with its own isolated **MongoDB** database.
- **Messaging Layer:** For asynchronous, event-driven communication, the system utilizes **RabbitMQ** as the central message broker, working in conjunction with a **Debezium** service for Change Data Capture (CDC).
- **Frontend Application:** The client-facing interface is a Single Page Application built with **Angular** and **PrimeNG**. It communicates exclusively with the BFF via GraphQL.

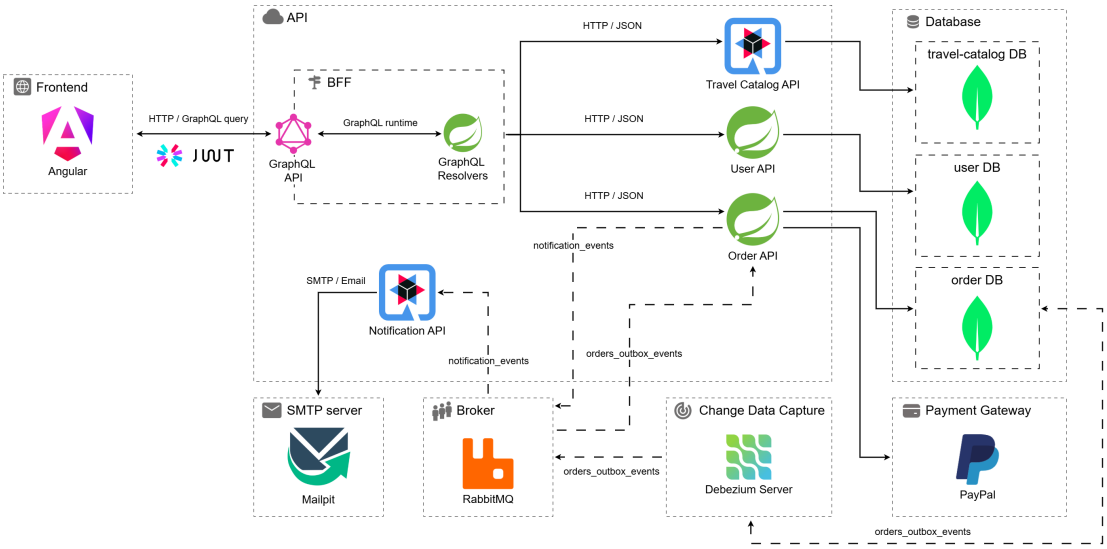

Figure 1: High-level system architecture of the Smart Travel platform, detailing the microservices topology, the API Gateway and the event-driven messaging layer.

The BFF and Distributed Data Consistency

To prevent frontend coupling and over-fetching, a **BFF** was implemented using Spring for GraphQL. It dynamically aggregates downstream data and serves as the primary security perimeter, enforcing Role-Based Access Control (RBAC) using JSON Web Tokens (JWT).

The most complex challenge was maintaining data consistency during the financial checkout phase. To solve this, the platform implements the **Transactional Outbox Pattern** combined with **Change Data Capture (CDC)**. When a user completes a payment, the event is saved atomically into MongoDB. A standalone **Debezium** service scans the database operations log (*oplog*) and publishes the event to RabbitMQ. Finally, the Notification API asynchronously consumes the event, guaranteeing at-least-once message delivery.

Infrastructure Automation and Cloud Deployment

To operationalize the platform, deployment was fully automated via **Jenkins Declarative Pipelines**. Ephemeral Docker build agents dynamically compile, containerize, and securely push optimized images to an internal **OKD (OpenShift)** ImageStream registry. The workloads are orchestrated via Kubernetes manifests (`Deployments`, `ConfigMaps`, `Secrets`). For security, the internal microservices are hidden from the public internet; instead, the Nginx frontend container acts as a secure Reverse Proxy routing traffic to the internal cluster network.

Personal Contribution and Results Obtained

The architectural design, software engineering, and pipeline automation of the Smart Travel platform were exclusively my personal contributions. Specifically:

- **System Engineering:** Designing the Spring and Quarkus microservices, including the GraphQL BFF and JWT security layer, and the frontend Angular SPA.
- **Distributed Systems Integration:** Implementing the Transactional Outbox Pattern, configuring Debezium and establishing the RabbitMQ event-driven architecture.
- **Frontend & DevOps:** Engineering all the Jenkins CI/CD pipelines and Kubernetes/OKD deployment files.

The project resulted in the successful deployment of a highly scalable, fault-tolerant cloud-native platform. The transition to a reactive architecture solved traditional blocking I/O limitations. The BFF eliminated frontend network inefficiencies, while the Debezium/RabbitMQ pipeline guaranteed full data consistency across distributed transactions. Ultimately, the thesis demonstrates that combining modern Java frameworks, reactive programming and DevOps automation successfully produces resilient software capable of meeting dynamic enterprise demands.